SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	G. Jaswanth	**Reg. No.:**	192472235
Section / Batch:	2024	**Date:**	

Code of Conduct

I G. Jaswanth - 192472235 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *G. Jaswanth*

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A conversational AI system must identify grammatical relationships in user queries containing long-distance dependencies. The developers are comparing constituency parsing and dependency parsing.

Analyze how the two representations differ in capturing hierarchical and word-to-word relationships, and justify which approach is more suitable for extracting meaningful relationships from conversational text.

2. An NLP system processes partially typed search queries such as “best hotels near…” and “book a flight from Chennai to…”. The system currently uses conventional top-down parsing, while the developers are considering Earley parsing.

Analyze how the two approaches behave when input is incomplete or ambiguous, and justify which parsing strategy is more appropriate for interactive NLP applications.

3. A grammar-checking system must distinguish between multiple interpretations of structurally ambiguous sentences. The developers are comparing CFG, PCFG, and neural constituency parsers.

Analyze how each approach represents and resolves syntactic ambiguity, and justify which approach would provide the best balance between interpretability and practical accuracy.

4. A multilingual NLP system must process sentences in which grammatical information depends on number, gender, tense, and person. The developers are considering feature structures and traditional context-free grammar rules.

Analyze how feature structures extend basic grammar representations and justify their usefulness for enforcing grammatical constraints in multilingual systems.

5. A voice assistant must interpret commands containing verbs with different argument requirements, such as “give the book to John,” “sleep,” and “put the file on the desk.” The developers are considering subcategorization frames.

Analyze how subcategorization information helps determine valid sentence structures and justify its importance in semantic and syntactic analysis.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1	4	4	4	3	3	18	92
2	4	4	4	3	3	18	
3	4	3	3	4	4	18	
4	4	4	4	4	3	19	
5	4	4	3	4	4	19	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Constituency Parsing vs Dependency Parsing

Scenario

A conversational AI system must identify grammatical relationships in user queries containing **long-distance dependencies**. The developers are comparing **constituency parsing** and **dependency parsing**.

The goal is to determine which representation is more suitable for extracting meaningful relationships from conversational text.

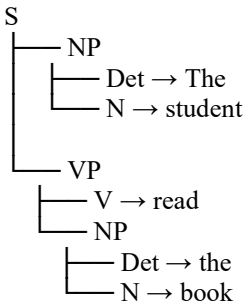
1. Constituency Parsing

Constituency parsing represents a sentence as a hierarchy of nested phrases or constituents.

For example:

"The student read the book."

A simplified constituency structure is:



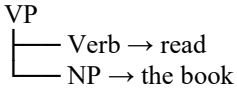
The sentence is divided into major constituents such as:

- **S** → Sentence
- **NP** → Noun Phrase
- **VP** → Verb Phrase
- **Det** → Determiner
- **N** → Noun
- **V** → Verb

What it captures

Constituency parsing is particularly good at showing **hierarchical phrase structure**.

For example:



This tells us that **"the book" forms a noun phrase functioning inside the verb phrase**.

2. Dependency Parsing

Dependency parsing represents a sentence through **direct word-to-word relationships**.

For:

“The student read the book.”

a simplified dependency structure is:

```
      read
     /    \
 student  book
    |      |
  The     the
```

The relationships can be represented as:

Head	Dependent	Relationship
read	student	subject
read	book	object
student	The	determiner
book	the	determiner

The important relationship is:

read→book=object

and:

read→student=subject

Thus, dependency parsing directly represents the **grammatical relationship between individual words**.

3. Major Difference

The fundamental difference is:

Constituency = phrase-to-phrase structure Dependency = word-to-word relationships

Comparison

Aspect	Constituency Parsing	Dependency Parsing
Basic representation	Nested phrases	Word-to-word dependencies
Main focus	Hierarchical structure	Grammatical relationships
Basic unit	Constituent/phrase	Word
Shows phrase boundaries	**Excellent**	Limited
Shows subject/object	Indirectly	**Directly**
Long-distance relationships	Can represent them hierarchically	**Directly represents dependencies**

Aspect	Constituency Parsing	Dependency Parsing
Semantic relationship extraction	Moderate	**Strong**
Conversational queries	Useful	**Highly suitable**
Interpretation	Phrase structure	Head-dependent relationships

4. Example with a Long-Distance Relationship

Consider:

> **“The book that the student borrowed yesterday was interesting.”**

This contains a long-distance relationship between:

```
book
↓
was interesting
```

and a relative clause:

```
the student borrowed [the book]
```

Constituency representation

A constituency parser emphasizes the nested structure:

```
S
├── NP
│   ├── The book
│   └── Relative Clause
│       ├── the student
│       └── borrowed ...
└── VP
    ├── was
    └── interesting
```

This clearly shows that the relative clause is embedded inside the noun phrase.

Dependency representation

Dependency parsing directly connects words according to their grammatical relationships.

Conceptually:

```
        interesting
            |
           was
            |
           book
          /    \
       The      borrowed
              /      \
          student   yesterday
```

The exact dependency structure depends on the parser and linguistic analysis, but the key advantage is that **relationships between the relevant words are represented directly rather than requiring traversal of nested phrase nodes**.

5. Which Is Better for Conversational Text?

For the specific requirement:

> **"extracting meaningful relationships from conversational text"**

Dependency parsing is generally the more suitable representation.

Reason 1 — Direct relationships

Conversational systems frequently need information such as:

Who performed the action?
What was affected?
Where?
When?
To whom?

Dependency parsing directly exposes relationships such as:

verb → subject
verb → object
verb → modifier

Reason 2 — Long-distance dependencies

When words that are grammatically related are separated by intervening words or clauses, dependency representations explicitly encode their relationship.

This makes them useful for extracting:

- Subjects
- Objects
- Modifiers
- Arguments
- Relations between entities

Reason 3 — Semantic information extraction

Suppose a user says:

> **"The customer who contacted the bank yesterday requested a new card."**

A conversational AI system may want:

Customer → requested → new card
Customer → contacted → bank
Time → yesterday

Dependency structures make these word-level relationships relatively direct to extract.

Reason 4 — Conversational queries are often short or incomplete

Users may say:

> "Book me a flight to Delhi."

or:

> "Hotels near Chennai airport."

The system is usually more interested in extracting:

Action → book
Object → flight
Destination → Delhi

rather than constructing a complete hierarchy of grammatical phrases.

Dependency parsing is therefore particularly useful for this type of semantic extraction.

6. However, Constituency Parsing Still Has Advantages

Dependency parsing is not universally better.

Constituency parsing is valuable when the system needs to understand **nested phrase structure**.

For example:

> **"The hotel near the airport that opened last year..."**

A constituency tree can clearly show which phrases belong inside which noun phrases.

Therefore:

Constituency is stronger for:

- Phrase structure
- Nested clauses
- Constituents
- Hierarchical syntactic analysis

Dependency is stronger for:

- Word-to-word relationships
- Subject/object extraction
- Semantic-role preparation
- Information extraction
- Conversational query understanding

7. Critical Comparison

The choice depends on the **application objective**.

If the system's primary goal were:

> “Understand the complete hierarchical grammatical structure of a sentence.”

then **constituency parsing** would be a strong choice.

However, the scenario specifically requires:

> **“extracting meaningful relationships from conversational text.”**

For this objective, the important information is usually the relationship between words and their grammatical roles.

Therefore:

Dependency parsing is the better choice for this scenario

8. Recommended Architecture

A practical conversational NLP system could use both approaches:

User Query
↓
Tokenization
↓
POS Tagging
↓
Dependency Parsing
↓
Relationship Extraction
↓
Semantic Representation
↓
Conversational Response

Constituency parsing can still be incorporated when **complex hierarchical or nested structures** need deeper analysis.

A combined architecture would therefore provide:

Constituency Parser
↓
Hierarchical Structure
↓
+
Dependency Parser
↓
Word Relationships
↓
Semantic Analysis

Conclusion

Constituency parsing and dependency parsing represent language in fundamentally different ways. **Constituency parsing focuses on hierarchical phrase structure**, while **dependency parsing focuses on direct relationships between words**.

For the given conversational-AI scenario, dependency parsing is more suitable because the system primarily needs to extract **subjects, objects, actions, modifiers, and other word-level relationships**, including relationships that may span long distances.

However, constituency parsing remains valuable for understanding complex nested phrase structures. Therefore, a practical NLP system can use **dependency parsing as the primary representation for relationship extraction**, while incorporating constituency analysis when deeper hierarchical structure is required.

Best choice for this scenario: Dependency Parsing

Q2. Top-Down Parsing vs Earley Parsing

Scenario

An NLP system processes partially typed search queries such as:

> **"best hotels near…"**

and

> **"book a flight from Chennai to…"**

The current system uses **conventional top-down parsing**, while the developers are considering **Earley parsing**.

The comparison should focus on **incomplete input, ambiguity, parsing efficiency, and suitability for interactive NLP applications**.

1. Top-Down Parsing

Top-down parsing begins with the **start symbol** of the grammar and tries to generate a structure that matches the input.

For example:

S
↓
NP VP
↓
...

The parser repeatedly expands grammar rules until it reaches the input words.

Basic process

Start Symbol
↓
Grammar Rule
↓
Expand Non-terminal
↓
Match Input
↓
Continue / Backtrack

For a complete sentence such as:

“I booked a hotel.”

top-down parsing can work effectively with a small and well-defined grammar.

2. Problem with Incomplete Input

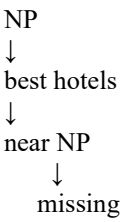
Consider:

“best hotels near…”

The user has not finished the query.

A conventional top-down parser may have difficulty because it is attempting to construct a complete grammatical structure.

It may reach a point where it expects additional constituents:



The input ends before the expected constituent is supplied.

Therefore, the parser may fail or need special handling for incomplete input.

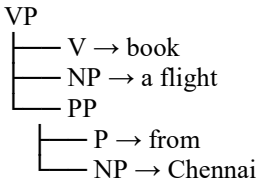
3. Problem with Ambiguous Input

Consider:

“book a flight from Chennai to…”

The parser has to determine what follows the preposition **“to.”**

The partial structure could be:



The parser is still waiting for the destination associated with:

to ...

A top-down parser may need to explore alternative grammar paths and potentially backtrack when the input does not match the expected structure.

4. Earley Parsing

Earley parsing is a **chart-based parsing algorithm for context-free grammars**.

It uses three major operations:

1. **Prediction**
2. **Scanning**
3. **Completion**

Basic process

Input
↓
Prediction
↓
Scanning
↓
Completion
↓
Chart of parsing states

Instead of repeatedly restarting or backtracking through the grammar, Earley parsing stores intermediate parsing states in a **chart**.

5. Handling Incomplete Queries

This is one of the major advantages of Earley parsing for the given scenario.

Consider:

"best hotels near…"

The parser can maintain the partial state indicating that a noun phrase is expected after **"near."**

Conceptually:

best hotels near
↓
Prepositional phrase started
↓
Expected → location/entity
↓
Input incomplete

The system can therefore maintain a meaningful partial interpretation rather than simply treating the input as an invalid sentence.

This is useful in interactive search because the user may still be typing.

6. Handling Ambiguous Input

For:

"book a flight from Chennai to…"

Earley parsing can maintain the parsing state as the user continues typing.

When the user enters:

“Delhi”

the parser can complete the structure:

```
Book
└── Flight
    ├── From → Chennai
    └── To → Delhi
```

The final semantic representation becomes:

BOOK(Flight, From=Chennai, To=Delhi)

This makes Earley parsing well suited to interactive language understanding.

7. Comparison

Feature	**Top-Down Parsing**	**Earley Parsing**
Starting point	Start symbol	Start symbol + chart states
Parsing style	Grammar expansion	Chart-based parsing
Incomplete input	**Limited**	**Better supported**
Ambiguous input	May require backtracking	Maintains multiple states
Backtracking	Can be significant	Avoids much repeated work
Partial structures	Limited	**Can maintain partial states**
CFG support	Yes	**General CFG**
Interactive search	Less suitable	**More suitable**
Implementation	Relatively simple	More complex
Real-time use	Depends heavily on grammar	Good for flexible CFG-based parsing

8. Efficiency Analysis

Top-Down Parsing

For simple, unambiguous sentences:

```
Small grammar
    ↓
Few alternatives
    ↓
Fast parsing
```

But ambiguity can cause:

```
Multiple grammar choices
      ↓
Backtracking
```

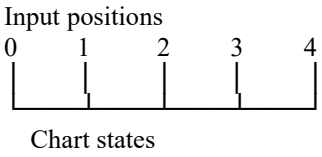
↓
Repeated work
↓
Slower processing

This can become problematic for interactive applications where the system must respond quickly.

Earley Parsing

Earley parsing stores intermediate results in its chart.

Conceptually:



Previously computed states can be reused rather than repeatedly recomputed.

This makes Earley parsing particularly useful when the grammar is ambiguous or the input contains complex structures.

9. Advantages of Top-Down Parsing

Top-down parsing is still useful.

Advantages

- Simple to understand
- Easy to implement
- Works well with small grammars
- Can be efficient for predictable input
- Suitable when grammar ambiguity is low

Limitations

- Backtracking can increase computational cost
- Incomplete input can be difficult
- Ambiguous structures can create many alternatives
- Less naturally suited to interactive partial queries

10. Advantages of Earley Parsing

Advantages

- Handles general CFGs
- Better suited to ambiguous structures
- Maintains partial parsing states
- Useful for incomplete input
- Reduces repeated parsing through chart states
- Appropriate for interactive NLP applications

Limitations

- More complicated than a basic top-down parser

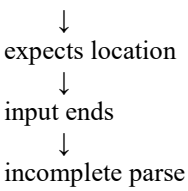
- Chart management introduces additional overhead
- Performance still depends on grammar complexity and ambiguity

11. Application to the Given Search Queries

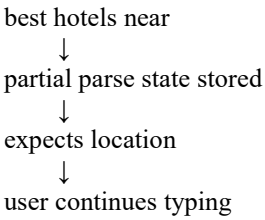
Query 1

“best hotels near…”

Top-down:



Earley:

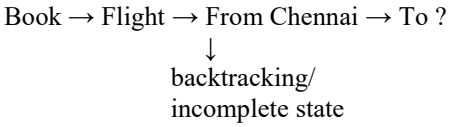


Therefore, Earley provides a better representation of the **current partial query**.

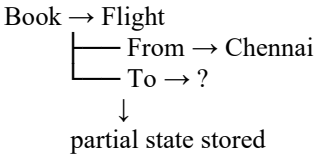
Query 2

“book a flight from Chennai to…”

Top-down:



Earley:



When the user types:

“Delhi”

the parser completes the intended relationship.

12. Critical Analysis

The correct choice depends on the application.

If the system processes only **complete, simple, grammatical sentences**, a conventional top-down parser can be sufficient because it is simpler and may be faster.

However, this system specifically processes **partially typed search queries**.

Interactive search requires the system to understand input **before the user has finished typing**.

Therefore, the ability to maintain partial parsing states becomes more important than implementation simplicity.

Interactive + incomplete + ambiguous input⇒Earley Parsing is more appropriate

13. Recommended Architecture

For an interactive search system:

User starts typing
↓
"best hotels near..."
↓
Tokenization
↓
Earley Parser
↓
Partial Parse
↓
Intent / Entity Extraction
↓
Search Suggestions
↓
User continues typing
↓
Updated Parse
↓
Final Search

This allows the system to provide suggestions even before the query is complete.

Conclusion

Top-down parsing is simple and effective for small, predictable, and complete inputs, but it can suffer from backtracking and has limited natural support for incomplete queries.

Earley parsing is better suited to the given interactive NLP scenario because it can handle **general context-free grammars, ambiguity, and partial input while maintaining intermediate parsing states**.

For queries such as:

“best hotels near…”

and:

“book a flight from Chennai to…”

the system needs to understand the query even before the user has completed it.

Therefore:

Earley Parsing is the more appropriate strategy for this scenario.

It provides a better balance between **flexibility, ambiguity handling, and interactive language understanding**, making it more suitable for real-time search applications.

Q3. CFG vs PCFG vs Neural Constituency Parser

Scenario

A grammar-checking system must distinguish between **multiple interpretations of structurally ambiguous sentences**. The developers are comparing:

1. **CFG — Context-Free Grammar**
2. **PCFG — Probabilistic Context-Free Grammar**
3. **Neural Constituency Parser**

The goal is to determine which approach provides the best balance between **interpretability and practical accuracy**.

1. Context-Free Grammar (CFG)

A CFG represents syntactic structure using a set of production rules.

For example:

S → NP VP
NP → Det N
NP → NP PP
VP → V NP
PP → P NP

Consider:

> **"I saw the man with a telescope."**

This sentence is structurally ambiguous.

Interpretation 1

I used a telescope to see the man.

VP
├── V → saw
├── NP → the man
└── PP → with a telescope

Here:

with a telescope
↓
modifies

↓
saw

Interpretation 2

I saw a man who had a telescope.

VP
├── V → saw
└── NP
  ├── the man
  └── PP → with a telescope

Here:

with a telescope
↓
modifies
↓
man

Important limitation

A CFG can generate **both valid parse structures**, but it does not inherently determine which one is more likely.

CFG identifies possible structures but does not rank them

2. Probabilistic Context-Free Grammar (PCFG)

A PCFG extends CFG by assigning probabilities to grammar rules.

For example:

VP → V NP PP 0.60
NP → NP PP 0.40

When an ambiguous sentence produces multiple parse trees, the parser can calculate the probability of each tree.

For:

"I saw the man with a telescope."

the system may obtain:

Parse 1 → 0.75
Parse 2 → 0.25

The system selects:

Parse 1

because it has the higher probability.

Advantage over CFG

CFG:

Parse A
Parse B

PCFG:

Parse A → 0.75
Parse B → 0.25

Therefore:

PCFG adds probabilistic ambiguity resolution

3. Neural Constituency Parser

A neural constituency parser uses a **machine-learning model** to predict syntactic structures from language data.

Instead of relying only on manually written grammar rules, it learns patterns from a training corpus.

Conceptually:

Input Sentence
↓
Word Representations
↓
Neural Model
↓
Candidate Constituency Structures
↓
Scoring
↓
Best Parse Tree

Modern neural parsers can use contextual representations to capture relationships based on the surrounding sentence.

For example:

"I saw the man with a telescope."

The model considers the surrounding context to determine whether:

with a telescope

is more likely associated with:

saw

or:

Man

4. Direct Comparison

Aspect	CFG	PCFG	Neural Constituency Parser

Aspect	CFG	PCFG	Neural Constituency Parser
Grammar representation	Explicit rules	Explicit rules + probabilities	Learned representations
Interpretability	**Very high**	**High**	Lower
Handles ambiguity	Generates alternatives	**Ranks alternatives**	**Learns contextual preferences**
Probability	No	**Yes**	**Yes, through model scores/probabilities**
Requires training corpus	Not necessarily	Usually for probabilities	**Yes**
Context sensitivity	Limited	Moderate	**Strong**
Adaptability	Low	Moderate	**High**
Explainability	**Excellent**	**Good**	More difficult
Practical accuracy	Depends heavily on grammar	Better than basic CFG	**Generally strongest with suitable training data**
Implementation complexity	Low	Moderate	**High**

5. Analysis of CFG

Advantages

- Simple and transparent
- Grammar rules are easy to understand
- Easy to inspect and modify
- Useful for controlled language
- Does not require large training datasets

Limitations

- Cannot naturally rank competing interpretations
- Ambiguity can produce multiple parse trees
- Limited ability to adapt to real-world language variation
- Requires substantial manual grammar design for complex language

Best suited for

Controlled / well-defined language
↓
Explicit grammar requirements
↓
CFG

6. Analysis of PCFG

Advantages

- Retains explicit grammar rules
- Adds probabilities to competing structures
- Better ambiguity resolution than CFG
- More interpretable than neural models
- Useful when syntactic structure needs to be explainable

Limitations

- Probabilities depend on the training data
- Basic PCFGs may not capture rich contextual information
- Rule probabilities alone may not fully represent long-range linguistic dependencies
- Requires probability estimation from a corpus

Best suited for

Need:
Grammar rules
+
Interpretability
+
Probabilistic ranking
↓
PCFG

7. Analysis of Neural Constituency Parsing

Advantages

- Learns patterns from data
- Strong contextual understanding
- Handles linguistic variation better
- Can resolve many ambiguities using surrounding context
- Usually more suitable for large-scale practical NLP systems

Limitations

- Requires suitable training data
- More computationally expensive
- More difficult to interpret than explicit grammar rules
- Errors can be harder to explain
- Model performance depends on training data and domain

For example, a neural model trained mainly on general English may perform differently when processing highly specialized legal or medical text.

8. Example Comparison

Consider:

> **“The student saw the teacher with a telescope.”**

There are two interpretations.

CFG

Produces:

Parse A

Parse B

but does not decide between them.

PCFG

Produces:

Parse A → 0.65
Parse B → 0.35

and selects Parse A.

Neural Parser

Uses contextual information from the sentence and learned linguistic patterns to assign scores to possible structures and select the most likely parse.

Therefore:

CFG
↓
Possible structures

PCFG
↓
Possible structures + probabilities

Neural Parser
↓
Contextual learned representation
↓
Best predicted structure

9. Which Approach Provides the Best Balance?

The question asks for the **best balance between interpretability and practical accuracy**.

CFG

Excellent interpretability but weaker ambiguity resolution.

Interpretability=High Practical flexibility=Lower

PCFG

Provides a strong middle ground.

It retains explicit grammar rules while adding probabilities for selecting among competing parses.

Interpretability=High Ambiguity handling=Better

Neural Constituency Parser

Typically offers stronger practical language understanding when trained appropriately, but its internal decisions are less transparent.

Practical accuracy=High Interpretability=Lower

10. Critical Evaluation

There is no single approach that is best for every application.

If the grammar-checking system prioritizes:

Maximum interpretability

then **CFG** is attractive.

If it requires:

Interpretability + probabilistic ambiguity resolution

then **PCFG** provides the strongest balance.

If it prioritizes:

Practical accuracy and contextual language understanding at scale

then a **neural constituency parser** is generally the stronger choice, assuming sufficient suitable training data and computational resources.

For the scenario specifically

Because the system is a **grammar-checking system** and the question explicitly asks for a balance between interpretability and practical accuracy, **PCFG is a strong balanced choice**.

It provides:

Explicit grammar rules
+
Probability-based ranking
+
Interpretable parse structures

Therefore:

PCFG provides the best balance for the stated requirement

11. Recommended Hybrid Approach

For a production grammar-checking system, an even stronger architecture could combine explicit and neural methods:

Input Sentence
↓
Neural Constituency Parser
↓
Candidate Structures
↓
CFG / Feature Constraints
↓
PCFG Probability Ranking
↓

Final Parse
↓
Grammar Checking

This approach can combine:

- **Neural models** → contextual understanding
- **CFG** → explicit grammatical constraints
- **PCFG** → interpretable probabilistic ranking

This hybrid design can provide both practical performance and stronger explainability.

Conclusion

CFG, PCFG, and neural constituency parsers differ mainly in how they handle syntactic ambiguity.

- **CFG** explicitly defines grammar but cannot inherently rank ambiguous interpretations.
- **PCFG** extends CFG with probabilities and can select the most likely parse while retaining understandable grammar rules.
- **Neural constituency parsers** use learned contextual representations and generally provide stronger practical language understanding, but with less transparency.

For the stated requirement of balancing **interpretability and practical accuracy**, **PCFG is the most suitable individual approach**.

Recommended choice: PCFG

Q4. Feature Structures vs Traditional Context-Free Grammar

Scenario

A multilingual NLP system must process sentences in which grammatical information depends on:

- **Number**
- **Gender**
- **Tense**
- **Person**

The developers are comparing **feature structures** with traditional **Context-Free Grammar (CFG)** rules.

The objective is to determine how feature structures extend basic grammar representations and why they are useful for enforcing grammatical constraints in multilingual NLP systems.

1. Traditional Context-Free Grammar

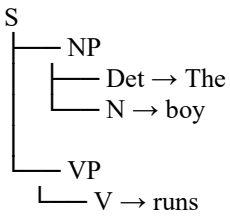
A CFG represents sentences using production rules such as:

S → NP VP
NP → Det N
VP → V NP

For example:

"The boy runs."

A simplified CFG structure is:



The CFG tells us that a sentence can contain:

NP + VP

However, basic CFG rules do not directly store detailed linguistic properties such as:

number = singular
gender = masculine
person = third
tense = present

2. Limitation of Basic CFG

Consider:

“The boy runs.”

This is grammatically correct.

But:

“The boy run.”

contains a subject–verb agreement problem in standard English.

A basic rule such as:

S → NP VP

does not itself specify that the NP and VP must agree in number.

Therefore, a traditional CFG may require many separate rules:

S → NP_singular VP_singular
S → NP_plural VP_plural

As the number of linguistic features increases, the grammar can become very large.

For example, we may need separate rules for combinations of:

Number
Gender
Person
Tense
Case

This causes **grammar-rule explosion**.

3. Feature Structures

A feature structure represents linguistic information as a collection of **attribute-value pairs**.

For example:

```
NP:
   number = singular
   person = third
   gender = masculine
```

A verb can have:

```
VP:
   number = singular
   person = third
   tense = present
```

The parser can then check whether the relevant features are compatible.

4. Example: Number Agreement

Consider:

> **"The boy runs."**

Feature representation:

```
NP
├── number = singular
└── person = third
```

Verb:

```
VP
├── number = singular
├── person = third
└── tense = present
```

Since:

```
NP.number = VP.number
```

the sentence satisfies the agreement constraint.

Therefore:

The boy runs. → Correct

Incorrect Example

> **"The boy run."**

Features:

Subject:
number = singular

Verb:
number = plural

Therefore:

singular ≠ plural

and the system identifies an agreement violation.

The boy run. → Incorrect

5. Feature Structures for Person

Consider:

“I run.”

The subject has:

number = singular
person = first

The verb is compatible with:

number = singular
person = first
tense = present

Now consider:

“He runs.”

The subject has:

number = singular
person = third

and the verb has:

number = singular
person = third
tense = present

Thus, feature structures allow the parser to distinguish between different grammatical forms.

6. Feature Structures for Gender

Gender becomes particularly important in languages where grammatical agreement depends on gender.

A simplified representation could be:

Noun:
  gender = feminine
  number = singular

An associated adjective could require:

Adjective:
  gender = feminine
  number = singular

The parser can enforce:

Noun.gender=Adjective.gender

and:

Noun.number=Adjective.number

This allows the grammar to represent agreement constraints that cannot be conveniently expressed using only a small set of ordinary CFG rules.

7. Feature Structures for Tense

Features can also represent tense.

For example:

“The student studied.”

The verb can be represented as:

Verb:
  tense = past
  number = singular
  person = third

For:

“The student studies.”

the representation can instead contain:

Verb:
  tense = present
  number = singular
  person = third

Thus:

Tense is represented explicitly

rather than being encoded only through separate grammar rules.

8. CFG vs Feature Structures

Aspect	Traditional CFG	Feature Structures
Basic representation	Production rules	Attribute-value pairs
Number	Limited	**Explicitly represented**
Gender	Difficult to encode efficiently	**Explicitly represented**
Person	Requires separate rules	**Explicitly represented**
Tense	Separate grammar rules	**Explicitly represented**
Agreement	Many rules may be required	**Feature unification/checking**
Grammar size	Can become large	More compact
Multilingual use	Difficult for rich agreement	**More flexible**
Constraint representation	Limited	**Strong**

9. Feature Unification

A major operation associated with feature structures is **unification**.

Unification attempts to combine two feature structures while ensuring that their shared features are compatible.

For example:

Subject:
number = singular
person = third

and:

Verb:
number = singular
person = third

can be unified because their values agree.

Result:

number = singular
person = third

Conflicting Features

Suppose:

Subject:
number = singular

and:

Verb:

number = plural

The values conflict:

singular ≠ plural

Therefore:

Unification fails

The sentence can be marked as grammatically inconsistent.

10. Why This Matters for Multilingual NLP

Different languages express grammatical information differently.

A language may require agreement based on:

- Number
- Gender
- Person
- Case
- Tense
- Animacy
- Other grammatical properties

A simple CFG can represent these distinctions, but doing so by creating separate rules for every combination can make the grammar extremely large.

Feature structures provide a more systematic approach.

For example:

NP
├── number = plural
├── gender = feminine
└── person = third

The same feature framework can be applied across different grammatical constructions.

11. Grammar Rule Comparison

Traditional CFG

A grammar may need rules such as:

S → NP_singular VP_singular
S → NP_plural VP_plural

and potentially more rules for different combinations.

Feature-Based Grammar

A single generalized rule can be associated with features:

S → NP VP

with a constraint such as:

NP.number = VP.number

Conceptually:

```
S
├── NP [number = X]
└── VP [number = X]
```

The same variable X ensures agreement.

This is much more compact.

12. Critical Analysis

Traditional CFG is useful when:

- The language is relatively simple.
- The grammar is small.
- Few agreement constraints exist.
- Explicit production rules are sufficient.

Feature structures are preferable when:

- The language contains rich grammatical agreement.
- Multiple features interact.
- The system is multilingual.
- Agreement constraints must be enforced.
- Grammar-rule explosion needs to be avoided.

Therefore, for the given scenario:

Feature Structures are more suitable

because the system explicitly needs to handle **number, gender, tense, and person**.

13. Recommended Architecture

A multilingual NLP system can use:

```
Input Sentence
    ↓
Tokenization
    ↓
POS / Morphological Analysis
    ↓
Feature Extraction
    ↓
Feature Structures
    ↓
Feature Unification
    ↓
Grammar Parsing
    ↓
Agreement Validation
    ↓
Semantic Analysis
```

For example:

"The boys run"

↓

Subject Features
number = plural
person = third

↓

Verb Features
number = plural
person = third

↓

Unification

↓

✓ Agreement satisfied

14. Advantages of Feature Structures

1. Compact representation

Instead of creating many separate grammar rules, features can store grammatical information directly.

2. Agreement checking

They allow the parser to verify:

number
gender
person

agreement.

3. Multilingual support

The same feature-based framework can represent different grammatical requirements across languages.

4. Better grammatical constraints

Features allow relationships between different parts of a sentence to be explicitly represented.

5. Reduced grammar-rule explosion

Instead of creating a separate rule for every combination, constraints can be expressed using feature values and unification.

15. Limitations

Feature structures also have some limitations:

- More complex than basic CFG.

- Requires appropriate feature definitions.
- Feature design may differ between languages.
- Large feature systems can increase implementation complexity.
- Feature structures alone do not solve every semantic or parsing problem.

Therefore, they are best used as part of a broader NLP parsing framework.

Conclusion

Traditional CFG represents grammatical structure mainly through production rules, but it does not efficiently represent detailed grammatical properties such as **number, gender, tense, and person**.

Feature structures extend CFG by representing these properties as **attribute-value pairs** and enforcing compatibility through **feature unification**.

For example:

Subject:
number = singular
person = third

Verb:
number = singular
person = third

can be successfully unified, while conflicting values indicate an agreement error.

Therefore:

Feature Structures are more suitable for the multilingual NLP scenario.

Q5. Subcategorization Frames in Voice Assistants

Scenario

A voice assistant must interpret commands containing verbs with different argument requirements, such as:

- **"Give the book to John."**
- **"Sleep."**
- **"Put the file on the desk."**

The developers are considering **subcategorization frames**.

The objective is to analyze how subcategorization information determines valid sentence structures and why it is important for syntactic and semantic analysis.

1. What is Subcategorization?

A **subcategorization frame** specifies the type and number of arguments that a verb requires to form a valid sentence.

Different verbs require different complements.

For example:

give → requires a giver, an object, and a recipient
sleep → requires only a subject
put → requires an object and a location

Therefore, the verb provides important information about the expected sentence structure.

2. “Give the book to John”

Consider:

“Give the book to John.”

The verb **give** requires:

Agent/Giver
Object/Theme
Recipient

The semantic representation is:

GIVE(Agent, Object, Recipient)

For the given sentence:

Agent → You / speaker
Object → book
Recipient → John

Therefore:

GIVE(You, Book, John)
Subcategorization frame
give → NP + NP + PP

Conceptually:

give
├── Object → the book
└── Recipient → to John

The frame tells the parser that **“to John” is not an arbitrary phrase**; it represents the recipient argument required by the verb.

3. “Sleep”

Now consider:

“Sleep.”

The verb **sleep** does not require an object.

A simple frame is:

sleep → Ø

or:

sleep → Subject

For example:

“The child sleeps.”

Subject → child
Action → sleep

Semantic representation:

SLEEP(Child)

An incorrect construction would be:

“The child sleeps the book.”

The verb **sleep** does not normally select a direct object.

Thus, subcategorization information can help identify that:

sleep + book
↓
invalid argument structure

4. “Put the File on the Desk”

Consider:

“Put the file on the desk.”

The verb **put** requires:

Agent
Object
Location

Semantic representation:

PUT(Agent, Object, Location)

For the sentence:

Agent → You
Object → file
Location → desk

Therefore:

PUT(You, File, Desk)
Subcategorization frame
put → NP + PP

where the PP provides the required location.

put
├── Object → the file
└── Location → on the desk

5. Comparison of the Three Verbs

Verb	Required Structure	Arguments	Example
give	Verb + Object + Recipient	Agent, Object, Recipient	Give the book to John
sleep	Verb only / no object	Agent	The child sleeps
put	Verb + Object + Location	Agent, Object, Location	Put the file on the desk

This demonstrates why a single generic rule such as:

VP → V NP

is not sufficient for all verbs.

Different verbs impose different requirements.

6. Why Subcategorization Is Important

1. Determines Valid Sentence Structures

The system knows what complements a verb expects.

For example:

give → object + recipient
put → object + location
sleep → no object

This allows the parser to distinguish valid and invalid structures.

2. Helps Semantic Role Assignment

Subcategorization provides clues about the semantic roles of the arguments.

For:

“Give the book to John.”

the system can determine:

give
├── book → Theme/Object
└── John → Recipient

For:

“Put the file on the desk.”

it can determine:

```
put
├── file → Theme/Object
└── desk → Location
```

Thus, syntax and semantics are closely connected.

7. Preventing Incorrect Interpretations

Consider:

> **“Put the book.”**

The system knows that **put** normally requires a location.

Its subcategorization frame is approximately:

put → Object + Location

But the input provides only:

Object → book
Location → missing

Therefore, the voice assistant should recognize that information is incomplete.

It could ask:

> **“Where should I put the book?”**

This is much better than executing an incomplete command.

8. Example of Ambiguity

Consider:

> **“Give John the book.”**

The system can interpret:

Agent → You
Recipient → John
Object → book

The surface order differs from:

> **“Give the book to John.”**

but the underlying semantic roles remain approximately the same:

GIVE(You, Book, John)

Subcategorization information helps the system understand that **give** can occur with different syntactic realizations while maintaining its argument structure.

9. Subcategorization vs Basic CFG

Basic CFG

A basic grammar might contain:

VP → V
VP → V NP
VP → V NP PP

These rules describe possible structures but do not necessarily specify which verbs allow which structure.

With Subcategorization

The grammar can associate verbs with their required frames:

give → NP PP
sleep → no complement
put → NP PP

Therefore, the parser can select structures based on the particular verb.

10. Comparison

Aspect	**Basic CFG**	**Subcategorization Frames**
Grammar structure	General	Verb-specific
Verb requirements	Limited	**Explicit**
Argument number	Not directly specified	**Specified**
Semantic roles	Indirect	**Better supported**
Invalid complements	Harder to detect	**Easier to detect**
Voice-command interpretation	Basic	**More accurate**
Missing arguments	Limited handling	**Can identify missing arguments**

11. Critical Analysis

A voice assistant cannot treat every verb in the same way.

For example:

sleep
→ no object required

give
→ object + recipient

put
→ object + location

If the system only uses surface syntax, it may incorrectly assume that every verb can take the same type of complement.

Subcategorization provides **lexical information about the verb**, allowing the parser to determine:

1. Which arguments are required.
2. Which arguments are optional.
3. What syntactic form those arguments can take.
4. What semantic role each argument is likely to have.

This makes it particularly useful for **semantic parsing of voice commands**.

12. Recommended Voice Assistant Architecture

A suitable system can use:

Voice Command
↓
Speech Recognition
↓
Tokenization / POS Tagging
↓
Verb Identification
↓
Subcategorization Frame
↓
Syntactic Parsing
↓
Argument Identification
↓
Semantic Role Assignment
↓
Intent Representation
↓
Execute / Ask Clarification

For example:

"Put the file on the desk."
↓
Verb = put
↓
Frame = Object + Location
↓
Object = file
Location = desk
↓
PUT(file, desk)

13. Handling Missing Information

The architecture can also detect incomplete commands.

Input

“Put the file.”

Analysis:

Verb → put
Object → file
Location → missing

The system should respond:

> **“Where should I put the file?”**

Another example

> **“Give the book.”**

Analysis:

Verb → give
Object → book
Recipient → missing

The system could ask:

> **“Who should I give the book to?”**

Thus, subcategorization enables **interactive clarification**.

14. Final Comparison of the Given Commands

Command 1

> **“Give the book to John.”**

GIVE
├── Agent → You
├── Object → book
└── Recipient → John

Command 2

> **“Sleep.”**

SLEEP
└── Agent → You

Command 3

> **“Put the file on the desk.”**

PUT
├── Agent → You
├── Object → file
└── Location → desk

Conclusion

Subcategorization frames provide **verb-specific information about the number and type of arguments required by a verb**.

In the given voice-assistant scenario:

GIVE→Object+Recipient SLEEP→No Object PUT→Object+Location

This information allows the system to determine valid sentence structures, identify missing or invalid arguments, and assign appropriate semantic roles.

Therefore, subcategorization frames are highly important for **syntax-driven semantic analysis and voice-command interpretation** because they connect the syntactic requirements of verbs with the semantic roles of their arguments.

Subcategorization is essential for accurate syntactic and semantic analysis of voice commands.
